REINFORCEMENT LEARNING · MID-REVIEW

SCoPe: Trust-Region Latent Planning with JEPA World Models

Plan in a learned world model, but act only on imagined futures the model can vouch for.

TEAM

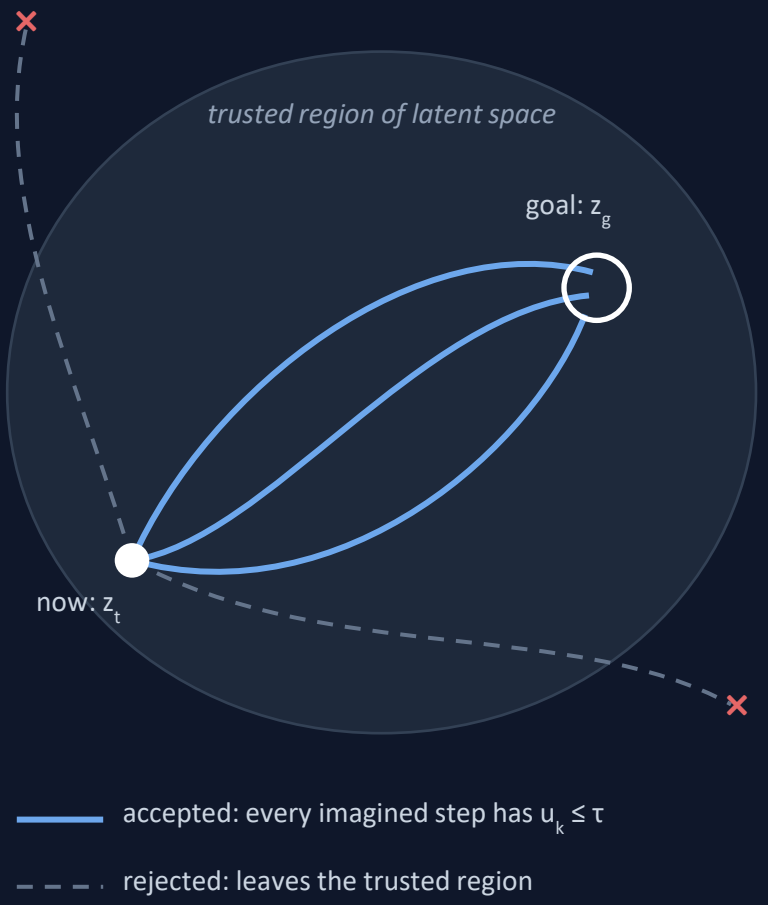
[TEAM MEMBERS]

GUIDE

[GUIDE NAME]

[DEPARTMENT] · [INSTITUTION] · [REVIEW DATE]

RL area model-based RL · planning with a learned latent world model (MPC)



OVERVIEW

The review covers the six required areas; every number is labelled

1 Title and team

Project, team, guide, RL problem area

Slide 1

2 Problem background and statement

A failure we measured, why it matters, the research question

Slides 3–6

3 Existing work, gap and our difference

13 studies compared, counter-evidence, gap and positioning

Slides 7–12

4 MDP, algorithm, data and simulation

Formulation, objective, CEM-MPC choice, simulators, protocol

Slides 13–18

5 System design and architecture

End-to-end, agent–environment, data-flow and deployment views

Slides 19–23

6 Novelty, demo, progress, next steps

Measured results, contribution matrix, progress, roadmap

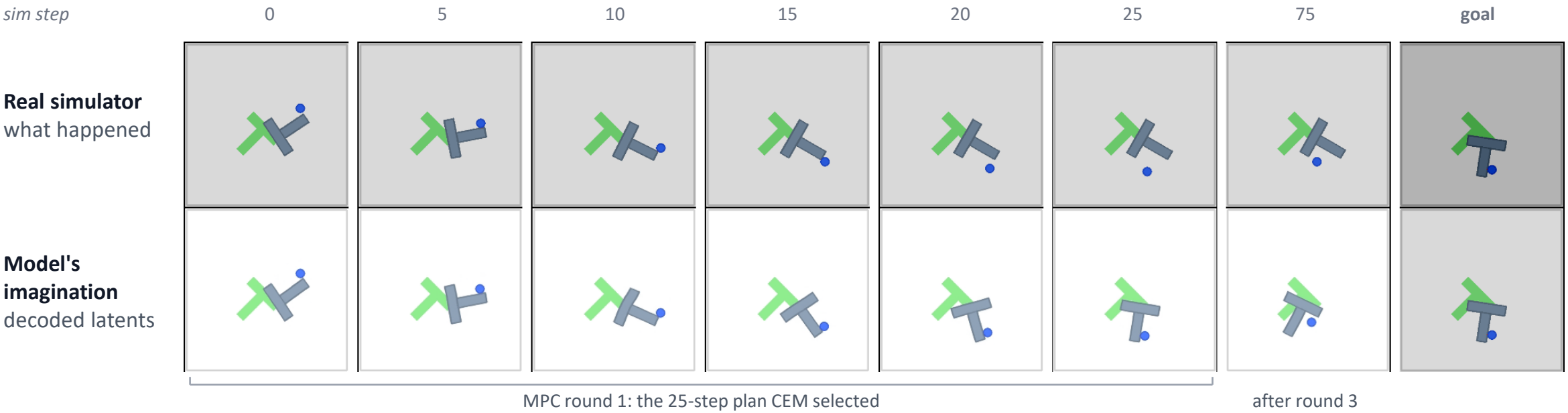
Slides 24–33

EVIDENCE LABELS

MEASURED	From our own runs; parsed from the logs by script (data/phase0_metrics.json)
ILLUSTRATIVE	Layout of a planned chart; contains no data
PUBLISHED	Reported by a cited paper [n]; not re-measured by us
TARGET	A planned objective that has not been achieved

MEASURED

A world model can imagine a success that never happens



Push-T · seed 101 · episode 0 · DINO-WM released checkpoint, upstream planner settings. The decoder is used only to visualise latents; planning happens in latent space.

1 Imagined

Of 300 candidate plans, CEM kept the one whose imagined end state was closest to the goal: the T turns toward the goal pose.

2 Real

In the simulator the same actions barely move the T. The episode was still unsolved after three replanning rounds.

3 Missing

Nothing in the planner asks whether an imagined future is reliable. Every rollout is trusted equally, right or wrong.

SCoPe's question: can we detect unreliable imagined rollouts and refuse to act on them, before they reach the real system?

PUBLISHED

Learned world models now plan real tasks, so their silent errors matter

80% vs 15%

Zero-shot pick-and-place (cup) on a real Franka arm: V-JEPA 2-AC planning with CEM in latent space vs the Octo policy baseline; world model trained on under 62 h of unlabelled robot video [2].

70% vs 0%

Hierarchical latent planning (HWM) vs single-level planning on real pick-and-place; compounding prediction error over long horizons is the bottleneck [15].

26% → 34%

Collision rate after adding a dynamics-uncertainty penalty to MPC in a navigation study, with uncertainty–risk correlation $r < 0.15$: the obvious fix can backfire [18].

WHY IT MATTERS

1 Errors become actions

A planner that exploits a model error executes it. On a robot that is a collision or a dropped object, not a lower score.

2 Safeguards sit at training time

Model-based RL truncates or penalises uncertain model rollouts while learning a policy [8–11]. The published latent planners rank candidate plans by predicted cost alone [1, 2].

3 Knowing when not to trust the model

lets a system re-plan, slow down or hand over instead of acting on a guess: a prerequisite for deployment.

Problem statement: plan only on imagined futures the model can vouch for

REAL-WORLD CHALLENGE

Robots that plan by imagining outcomes in a learned world model act on predictions that can be confidently wrong (slide 3).

EXISTING LIMITATION

Latent planners (DINO-WM, V-JEPA 2-AC) rank candidate plans by predicted cost alone. MBRL's uncertainty tools were built for low-dimensional states and training-time rollouts.

SEQUENTIAL CHALLENGE

An action's effect shows up steps later. Errors compound over the 5-step (25 simulator-step) horizon, and the whole plan runs before any feedback arrives.

PROPOSED APPROACH

Score every imagined step (u_k). Reject candidate plans that leave the trusted region (any $u_k > \tau$). Fall back: resample, shorter horizon, safe action, stop and flag.

MEASURABLE OBJECTIVE

Calibration of u against real latent error (AUROC, Spearman ρ, ECE); false rejection and false acceptance rates; success per MPC round against plain MPC, random and oracle gates.

SCOPE AND CONSTRAINTS

In: DINO-WM checkpoints on Push-T, Wall and PointMaze (simulated), frozen encoder, one 8 GB GPU. Out: real robots, retraining the world model, reward learning.

One research question, tested as four falsifiable hypotheses

PRIMARY RESEARCH QUESTION

Can an uncertainty score on imagined JEPA latents flag unreliable rollouts well enough that rejecting them improves real planning success, without rejecting most good plans?

H1 Calibration

Does u_k rank imagined steps by their real latent error?

Metric AUROC / AUPRC for top-10% error; Spearman ρ; ECE

Control random score; distance to start latent

H2 False rejection

How often does the gate refuse plans that would have worked?

Metric FRR = P(reject | real success)

Control random gate at equal rejection rate

H3 False acceptance

How often does the gate accept plans that fail?

Metric FAR = P(accept | real failure)

Control random gate at equal rejection rate

H4 Closed-loop benefit

Does gating raise success per MPC round?

Metric success rate per round; compute per episode

Control plain MPC, soft penalty, random and oracle gates

OBJECTIVES

O0 Reproduce DINO-WM planning on our GPU
done for Push-T

O1 Log imagined vs real latents (error labels)
Phase 1

O2 Test ≥ 3 scores offline (H1–H3)
Phase 2

O3 Gate the closed loop (H4)
Phase 3

O4 Second backbone (V-JEPA 2-AC)
stretch

Expected benefits, not results yet: fewer confidently wrong actions, an explicit stop-and-flag signal, and less compute spent on plans the model cannot evaluate.

PUBLISHED

Model-based RL already guards against model error, mostly in training

Study	How model error is handled	Setting	Reported finding	Limitation for SCoPe
PETS [6] Chua et al., NeurIPS 2018	Probabilistic ensemble; trajectory sampling inside CEM-MPC	MuJoCo, low-dimensional state	Model-free asymptotic return with 8× / 125× fewer samples than SAC / PPO (half-cheetah)	Uncertainty is propagated, never used to refuse a plan
MBPO [8] Janner et al., NeurIPS 2019	Short model rollouts branched from real data	MuJoCo	More sample-efficient than prior MBRL; matches the best model-free asymptotes	One global rollout length; trains a policy
MOPO [9] Yu et al., NeurIPS 2020	Reward penalised by model uncertainty	Offline benchmarks (D4RL)	Beats prior offline model-free and model-based baselines	Soft penalty in training; becomes our soft-penalty control
MOReL [10] Kidambi et al., NeurIPS 2020	Pessimistic MDP: unknown state-actions lead to an absorbing HALT state	Offline benchmarks (D4RL)	Matches or exceeds offline state of the art at publication	Hard cut-off, but at training time and in state space
MACURA [11] Frauenknecht et al., 2024 (preprint)	Rollout length adapted to local model uncertainty: "where to trust"	MuJoCo	Better data efficiency than prior deep MBRL	Training-time rollouts on proprioceptive state
Latent safety filter [16] Seo et al., CoRL 2025	Epistemic uncertainty with a conformal threshold marks OOD latents unsafe; reachability filter	Vision-based Franka arm, sim and hardware	Pre-emptively detects unsafe OOD scenarios (qualitative in the abstract)	Avoids failure sets with a learned safety value; not goal-reaching plans

What we take: ensembles and MC-dropout as candidate scores [6, 22, 23]; the soft penalty as a control [9]; the hard "unknown region" cut-off [10, 11]; conformal calibration of the threshold [16].

PUBLISHED

Latent planners scale, yet rank every imagined plan as equally reliable

Study	World model and planner	Setting	Reported finding	Reliability check at decision time
PlaNet [7] Hafner et al., ICML 2019	RSSM learned from pixels; CEM in latent space	DeepMind Control, pixels	Far fewer episodes than strong model-free agents, similar final return	None: every imagined rollout is trusted
DreamerV3 [12] Hafner et al., Nature 2025	RSSM; actor-critic trained in imagination	150+ tasks; Minecraft diamonds from scratch	One fixed configuration across domains	None; needs reward
TD-MPC2 [13] Hansen et al., ICLR 2024	Decoder-free latent model; MPPI with a learned value	104 continuous-control tasks	One 317M-parameter agent across 80 tasks	None; needs reward
DINO-WM [1] Zhou et al., ICML 2025	Frozen DINOv2 patch features + ViT predictor; CEM-MPC to a goal image	6 environments incl. PointMaze, Wall, Push-T; offline, reward-free	MPC success 0.98 / 0.96 / 0.90 on PointMaze / Wall / Push-T	None: its CEM returns every plan as valid. **Our base model.**
PLDM [14] Sobal et al., NeurIPS 2025	JEPA latent dynamics, used for planning	Navigation; offline data of varying quality	Generalises to unseen layouts better than model-free offline RL	None
V-JEPA 2-AC [2] Assran et al., 2025 (preprint)	Action-conditioned predictor on frozen V-JEPA 2; CEM energy minimisation	Real Franka arm, zero-shot	Pick-and-place 80% (cup) vs Octo 15%; about 16 s per action	None. **Our second backbone (stretch).**
HWM [15] Zhang et al., 2026 (preprint)	Multi-scale latent models; hierarchical MPC	Real Franka; simulated push and maze	70% vs 0% (single level) on real pick-and-place; up to 3× less planning compute	None; tackles horizon, not reliability

Pattern: strong planners, reward-free goal reaching in the JEPA family, and no decision-time reliability signal in any of them. That is the gap we test (slide 10).

PUBLISHED

Recent work warns that model uncertainty may miss the errors that matter

Uncertainty can shrink while error grows

Biased Dreams [17] · RLC 2026

In Dreamer's RSSM, ensemble disagreement tracks local error but not compounding error; rollouts drift into well-supported "attractor" regions where uncertainty looks low.

SCoPe response: test scores against real multi-step error at every imagined step (H1), not one-step disagreement.

Uncertainty penalties can raise risk

Learning from World Feedback [18] · ICML 2026 workshop

A dynamics-uncertainty penalty raised collisions from 26% to 34%; uncertainty–risk correlation $r < 0.15$; outcome-based signals cut collisions to 1–14%.

SCoPe response: include an oracle gate and a soft-penalty control, and report negative results.

Latent distance is not task distance

Representation geometry [20] · preprint 2026

Collapse-free JEPA latents need not reflect task-relevant distances, so latent planning costs can misjudge closeness to the goal.

SCoPe response: log two labels per candidate: latent error and task consequence.

Internal shifts do not predict outcomes

Beyond Task Success [21] and ACPC [19] · preprints 2026

Stage-wise analysis: the ranking of degradations by representation shift does not match their ranking by physical outcome. ACPC diagnoses JEPA models via perturbation consistency.

SCoPe response: judge every score at the outcome stage; ACPC-style consistency is a candidate score.

Our position: a trust score is a hypothesis to falsify. Calibration first (H1), then closed-loop benefit (H4) with an oracle gate as the ceiling.

The gap: rejecting unreliable imagined latents at decision time

EXISTING APPROACHES	DEMONSTRATED STRENGTHS	REMAINING LIMITATION	OUR INTERVENTION	HOW WE MEASURE IT
Uncertainty-aware model-based RL PETS, MBPO, MOPO, MOReL, MACURA [6, 8–11]	Ensembles, short or adaptive rollouts and penalties curb model exploitation; sample-efficient and safer offline.	Built for low-dimensional states and training-time rollouts. Disagreement can miss compounding latent error [17]; penalties can raise risk [18].	Per-step trust score u_k on imagined latents Hard gate inside CEM-MPC: reject a plan if any $u_k > \tau$ Fallback ladder when too few plans pass No retraining of the world model	**H1** AUROC, Spearman ρ, ECE against real latent error **H2–H3** false rejection and acceptance vs a random gate **H4** success per MPC round vs plain MPC, soft penalty, random and oracle gates
Latent world-model planners DINO-WM, V-JEPA 2-AC, PLDM, HWM [1, 2, 14, 15]	Reward-free, zero-shot planning to a goal image from offline data; real-robot success.	Every imagined rollout is treated as equally reliable. Confident failures like slide 3 go unflagged.		

Contribution type: an empirical test plus an integration (a gate inside an existing planner). Not a new world model and not a new RL algorithm.

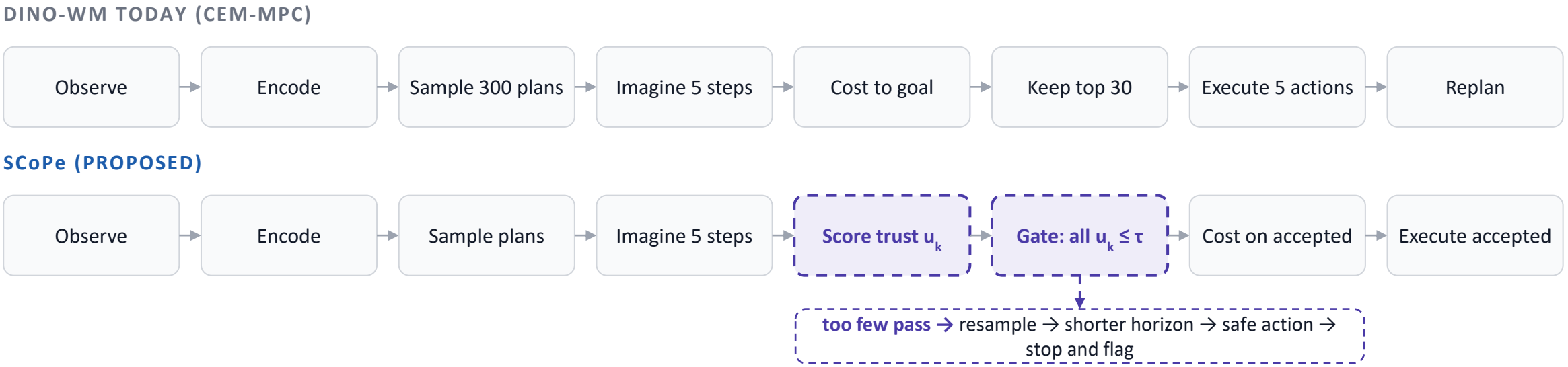
PUBLISHED PROPOSED

Where SCoPe sits: latent planning plus a decision-time trust check

	Model uncertainty not used	Uncertainty used at training time	Uncertainty used at decision time (can veto actions)
Low-dimensional state models	*(not our focus)*	MBPO [8] · MOPO [9] · MOReL [10] · MACURA [11]	PETS [6] — *propagates uncertainty through MPC; no veto*
Image-based latent world models	PlaNet [7] · DreamerV3 [12] · TD-MPC2 [13] · DINO-WM [1] · PLDM [14] · V-JEPA 2-AC [2] · HWM [15]	*analysed, not used for control: ensemble RSSMs in Biased Dreams [17]*	Latent safety filter [16] — *vetoes unsafe actions via a learned reachability value* **SCoPe (proposed)** — *vetoes candidate plans while reaching a goal image*

Closest neighbour: the latent safety filter [16] also acts on latent uncertainty at decision time, but to avoid failure sets. SCoPe filters goal-reaching plans and first tests the score against real error.

SCoPe's one change: check whether each imagined future is trustworthy



Dashed violet = new in SCoPe (planned). Imagined latents are never used as the start of the next plan; every replan starts from a real observation.

C1 Error-labelled dataset

Imagined vs real latents per candidate; latent error and task consequence; 4 sampling contexts

PLANNED Phase 1

C2 Trust-score calibration

≥ 3 scores on frozen-encoder JEPA latents against real error, with random and trivial controls (H1–H3)

PLANNED Phase 2

C3 Gated planner + fallback

Compared with plain MPC, soft penalty, random and oracle gates on identical start/goal pairs (H4)

PLANNED Phase 3

C4 Interfaces, 8 GB recipe

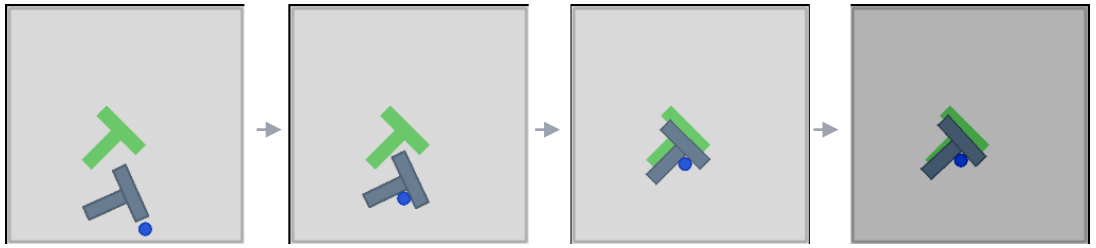
Backbone-agnostic contracts; DINO-WM planning 3.8× faster with identical CEM decisions

BUILT done

MEASURED PUBLISHED

Sequential decisions: we plan with a learned model, not a trained policy

DELAYED CONSEQUENCES IN PUSH-T



step 0
agent moves toward the T

step 15
contact: push rotates and slides it

step 25
judged only here

goal
pose within 20 px and 20°

Push-T seed 100, episode 3 (simulator frames from our run). Success is checked once, after the 25-step plan; no intermediate reward guides the choice of each push.

Why this is RL: an agent chooses actions to optimise a long-horizon objective under (learned) dynamics, and each action changes what is reachable next. **Which kind:** model-based RL at decision time. The dynamics model is learned offline from reward-free data; the "policy" is the planner (MPC) re-solved at every step.

Approach	What it needs	Fit for our setting
Behaviour cloning	goal-directed demonstrations	copies demonstrations; a new goal image needs new demos
Hand-written controller	state estimation and tuned contact rules	per-task engineering; no image goals
Model-free RL (PPO, SAC)	a reward and many online interactions	our data has no reward and is offline
Offline RL	reward labels in the dataset	reward-free data; one goal per policy
Planning with a learned world model (MPC)	offline trajectories and a goal image	reward-free, zero-shot to new goals, replans from feedback: DINO-WM Push-T 0.90 [1]

The planning approach is the one DINO-WM and V-JEPA 2-AC use [1, 2]; SCoPe changes how candidate plans are filtered, not the learning paradigm.

MEASURED

A goal-conditioned POMDP, planned as an MDP in latent space

Real system (partially observed)
state s: agent x, y; block x, y, θ; velocities
observation o = 224×224 RGB image + proprio
$s' \sim P(s' \mid s, a)$: pymunk physics, unknown to us

Latent MDP used for planning
$M = (Z, A, f_\theta, c_g, \gamma = 1, H = 5)$
z = enc(o): 196 patch tokens × 384 dims
$\hat{z}_{k+1} = f_\theta(z_{k-2..k}, a_k)$: ViT predictor, 3-frame history
a_k: 2-D action × 5 sim steps = 10-D chunk
c_g: terminal latent distance to goal (next slide)

Policy and return
$\pi(o_t, g)$: first 5 chunks of argmin over CEM plans of c_g, then replan (MPC)
Return = $-c_g$ of the final imagined step; γ = 1 on a 5-step horizon

Element	Project-specific definition	Push-T example
State / observation	Image + proprio (agent position); latent z from the frozen DINOv2 ViT-S/14 encoder [3]	T pose and agent visible in a 224×224 frame
Action	2-D relative target for the agent, tracked by a PD controller; planned in 10-D chunks (frameskip 5), normalised	push up-left by a few pixels per step
Reward / cost	No environment reward during planning; cost = latent distance of the last imagined step to the goal	goal = dataset state 25 steps ahead
Transition	Real: 2-D physics with contacts and friction. Imagined: ViT predictor (6 layers, 16 heads)	T rotates when pushed off-centre
Episode	One start/goal pair; MPC rounds of 25 sim steps, at most 3 rounds in our runs	≤ 75 sim steps
Termination	Success: $\|\Delta(\text{agent, block position})\| < 20$ and $\|\Delta\theta\| < \pi/9$, checked after each round; else stop at the round cap	Wall: Δpos < 4.5; PointMaze: < 0.5
Initial states	50 start/goal pairs per seed from the validation split (Push-T) or sampled states (Wall, PointMaze)	seeds 99, 100, 101

Values read from the DINO-WM code and configs we run (commit 0a9492f): plan_pusht.yaml, pusht_wrapper.eval_state.

MEASURED PROPOSED

Objective: a latent goal-distance cost, plus a trust constraint

$$\text{minimise over } a_t \dots a_{t+4}: \quad c_g = \mathrm{MSE}_{\text{visual}}(\hat{z}_{t+5}, z_g) + \alpha \cdot \mathrm{MSE}_{\text{proprio}}(\hat{z}_{t+5}, z_g)$$

$$\text{subject to (SCoPe):} \quad u_k \le \tau \text{ for every imagined step } k = 1 \dots 5; \quad \text{too few candidates pass} \rightarrow \text{fallback ladder}$$

Visual term

BUILT

Meaning Mean squared distance between predicted and goal DINOv2 patch tokens (N = 196 × 384).

Why The goal is given as an image; no reward model is needed.

Effect Pulls the imagined scene toward the goal scene.

Scale Feature units, no physical unit.

Risk Latent distance may not equal task distance [20]; the planner can exploit predictor errors (slide 3).

Proprio term (α)

BUILT

Meaning Squared distance between predicted and goal proprio embeddings (agent position).

Why Disambiguates where the agent is.

Effect Pulls the agent toward its goal position.

Scale α = 1 for Push-T and Wall, 0 for PointMaze: upstream values, not tuned by us.

Risk May favour the agent's position over the block's pose.

Trust constraint

PLANNED

Meaning Every imagined step must have $u_k \le \tau$, where u_k is the trust score (higher = less trusted).

Why Do not optimise over predictions the model cannot vouch for.

Effect Removes candidates; fallback if too few remain.

Scale τ = a quantile of u on held-out decision points, never on test episodes.

Risk Over-rejection (H2) or false confidence (H3).

Reward hacking here = model exploitation: CEM finds actions whose imagined latent matches the goal while the real one does not. Validation is always the simulator's success test, never the cost.

PUBLISHED

Algorithm: CEM-based MPC, chosen over gradient planning and policies

Criterion	CEM-MPC (chosen)	Gradient-descent planning	Policy learning (PPO, SAC; Dreamer)
What is optimised	300 sampled action sequences; refit mean and std to the top 30 each iteration	one action sequence by backprop through the predictor	policy network weights from reward
Learning type	none at test time: zero-shot planning on an offline-trained model	none at test time	on-policy (PPO), off-policy (SAC), in imagination (Dreamer)
Actions	continuous, 5 × 10-D chunks	continuous	continuous (PPO also discrete)
Needs	a goal image only	a goal image; differentiable model	a reward and many environment steps
Push-T, published [1]	**0.86 single plan; 0.90 with replanning**	0.28	not comparable: needs reward
Fit for a trust gate	explicit candidate set: reject before choosing elites	one trajectory, nothing to reject	a gate would need a model anyway
Main limitation	decision-time compute (≈ 3.4 min per plan on our GPU); exploits model error	local minima; adversarial action gradients	reward design; retraining per goal

Decision: keep DINO-WM's CEM-MPC (horizon 5, 300 samples, top 30, 30 iterations on Push-T) so our baseline equals the published one. MPPI, used by TD-MPC2 [13], is a weighted variant of the same sampling idea; the gate would apply unchanged. The algorithm is not our hypothesis, so no selection experiment is needed.

MEASURED PUBLISHED

Data and simulation: three simulated tasks, reward-free offline data

Task	Simulator	Offline data (published size)	Start / goal for evaluation	Success test	Status here
Push-T	pymunk 2-D physics; task from Diffusion Policy [24]	18.5k samples of noisy pushing trajectories [1]	validation-trajectory state 25 steps ahead	$\Delta pos < 20$ and $\lvert\Delta\theta\rvert < \pi/9$	**full runs, 3 seeds**
Wall	custom 2-D room with a door [1]	1,920 trajectories × 50 steps	random reachable states, seeded	$\Delta pos < 4.5$	**2+ seeds, 3 rounds**
PointMaze	MuJoCo 2.1 via mujoco-py (D4RL maze [25])	2,000 random trajectories	random reachable states, seeded	$\Delta pos < 0.5$	**full run, 1 seed**

Source and licence

DINO-WM code (MIT licence) pinned at commit 0a9492f; datasets and checkpoints from the authors' OSF project [26]. Archives are size-capped and SHA-256-checked before unpacking; the data licence is not stated on OSF.

Processing and state

Frames resized to 224 × 224 and normalised as upstream; actions and proprio normalised with dataset statistics; latent state = frozen DINOv2 patch tokens + proprio embedding, 3-frame history.

Splits and leakage

Upstream 90 / 10 train / validation split (seeded). The world model was trained on the train split; evaluation pairs come from validation trajectories or freshly sampled states.

What this data is not

Simulated, synthetic play data. No result here implies real-robot performance; real-world transfer is out of scope and would need V-JEPA 2-AC-style real data.

Protocol: what is trained, optimised and evaluated, without leakage

EVALUATION LOOP (EVERY METHOD, IDENTICAL SETTINGS)



TRAINED (offline, before planning)

World model: by the DINO-WM authors; we reuse it frozen

Trust scorers: Phase 2: ensemble heads on cached encoder features, kNN index; MC-dropout and self-consistency need no training

OPTIMISED (at decision time)

Action sequences: CEM refits a Gaussian over 300 candidates per iteration; no weights change

Stopping: 30 CEM iterations (early stop only if every episode already succeeds); at most 3 MPC rounds

EVALUATED

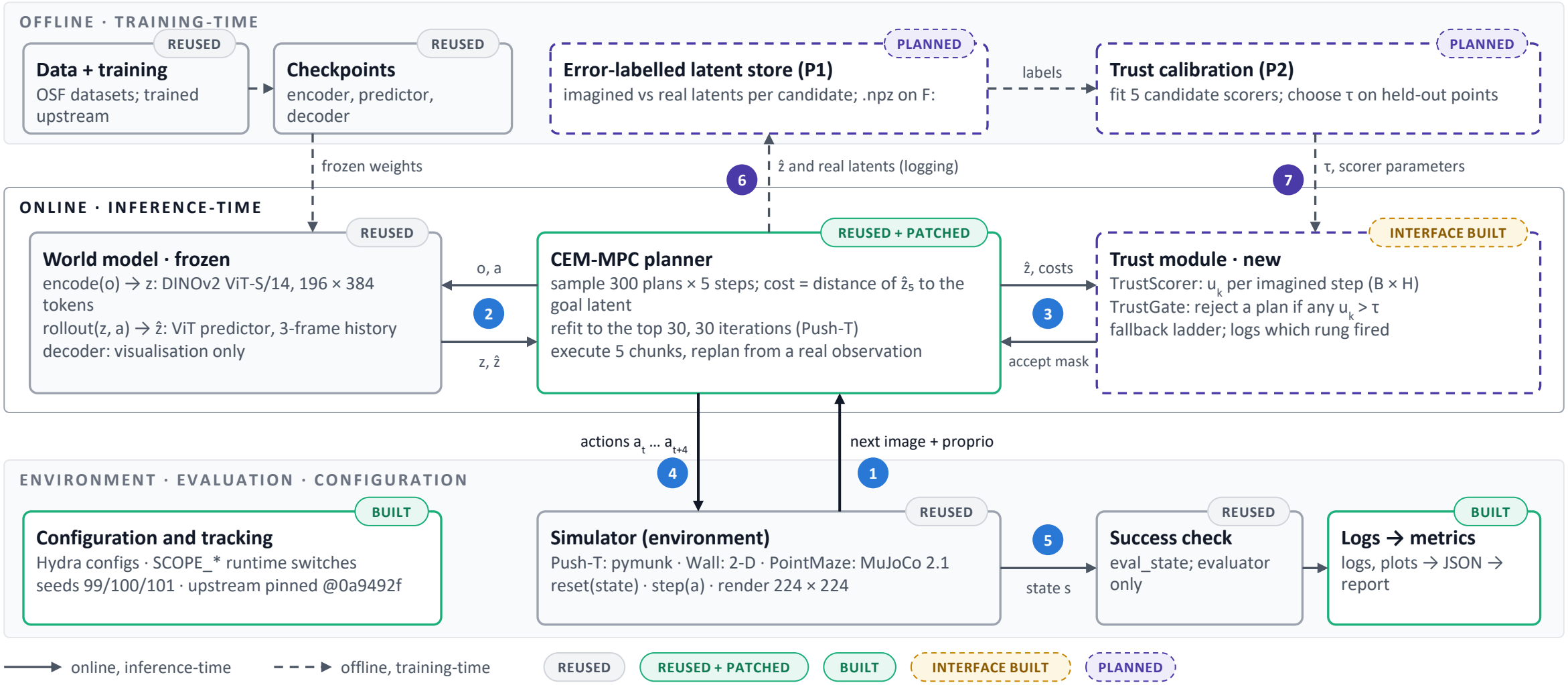
Phase 0: success per round vs published (done for Push-T)

Phases 2–3: H1–H3 offline on logged decision points; H4 in the closed loop

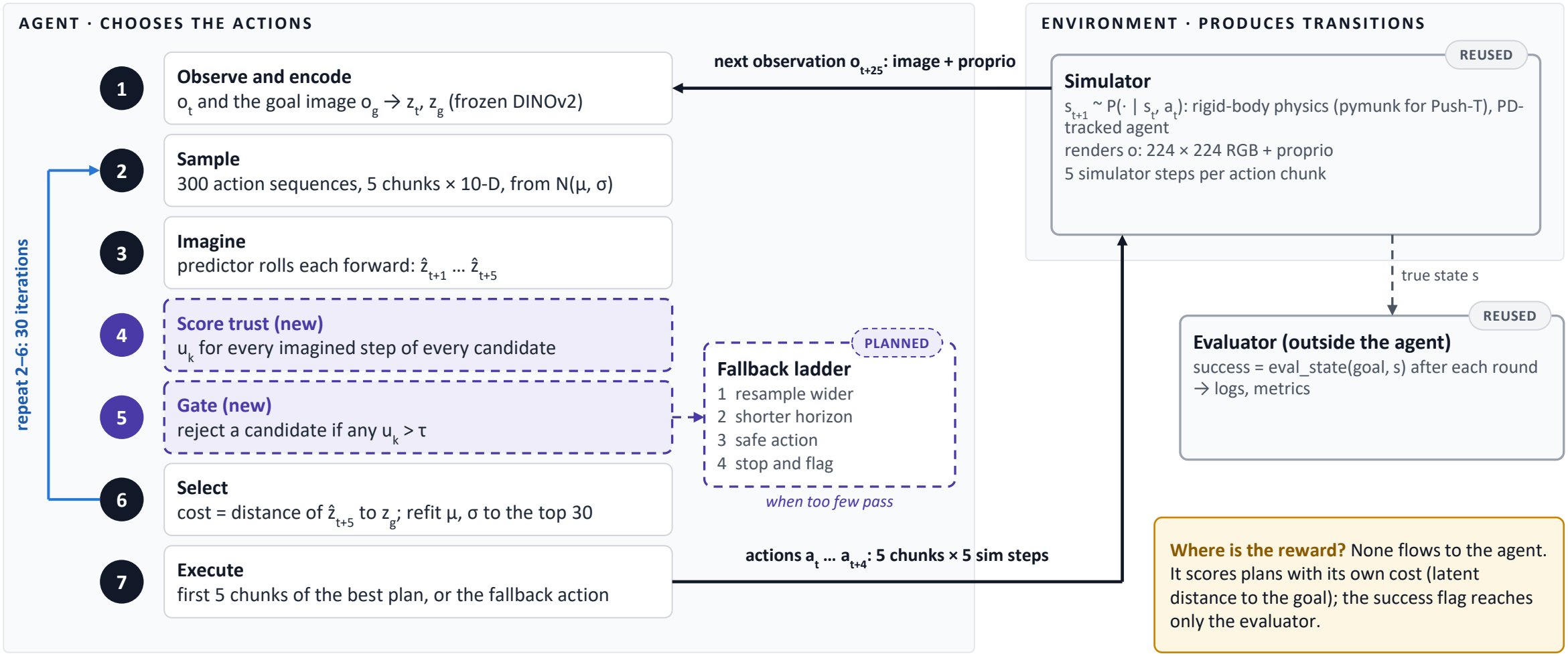
LEAKAGE CONTROLS

- Evaluation pairs never come from the world model's training split.
- τ, λ and scorer parameters are fit on decision points from separate calibration seeds; test episodes are never used for tuning.
- Every method sees the same seeds, start/goal pairs and CEM budget; the success flag reaches only the evaluator and upstream's stop rule, applied to all methods alike.

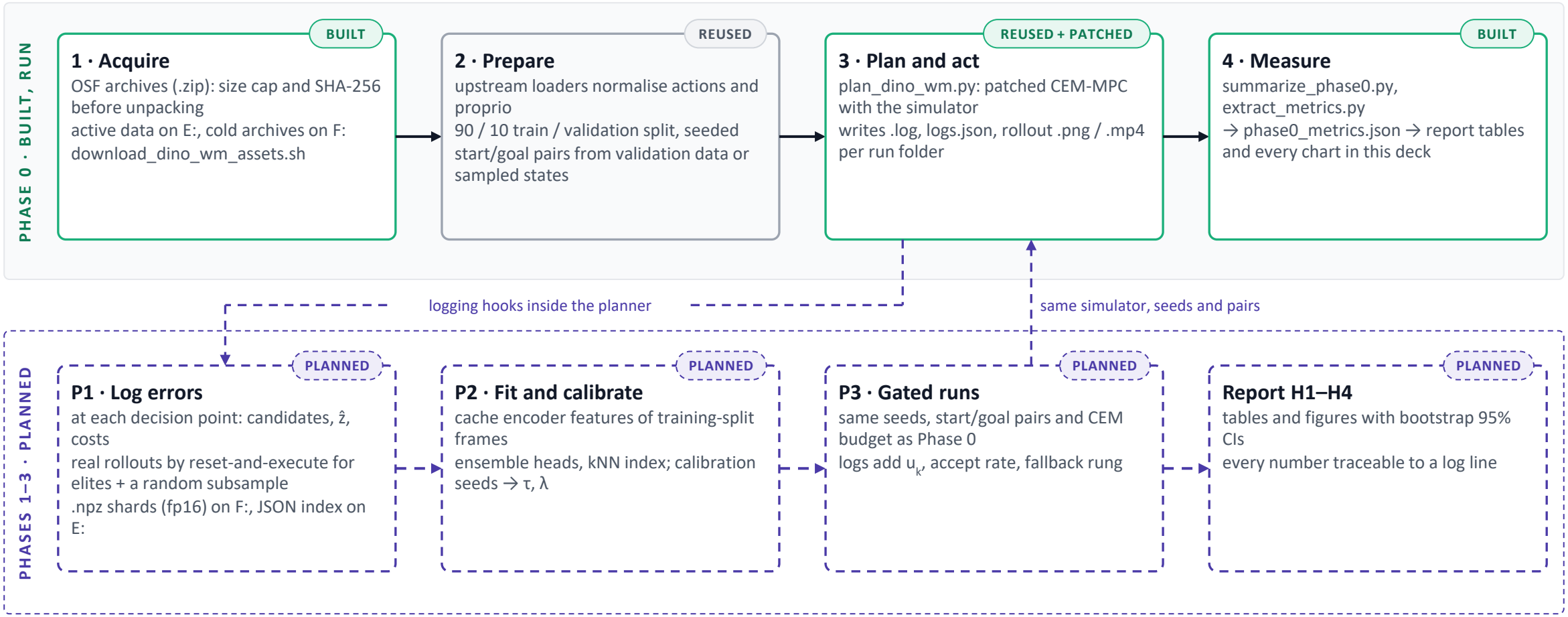
End-to-end architecture: online agent and offline learning loop



Agent–environment loop: the planner acts, the simulator transitions



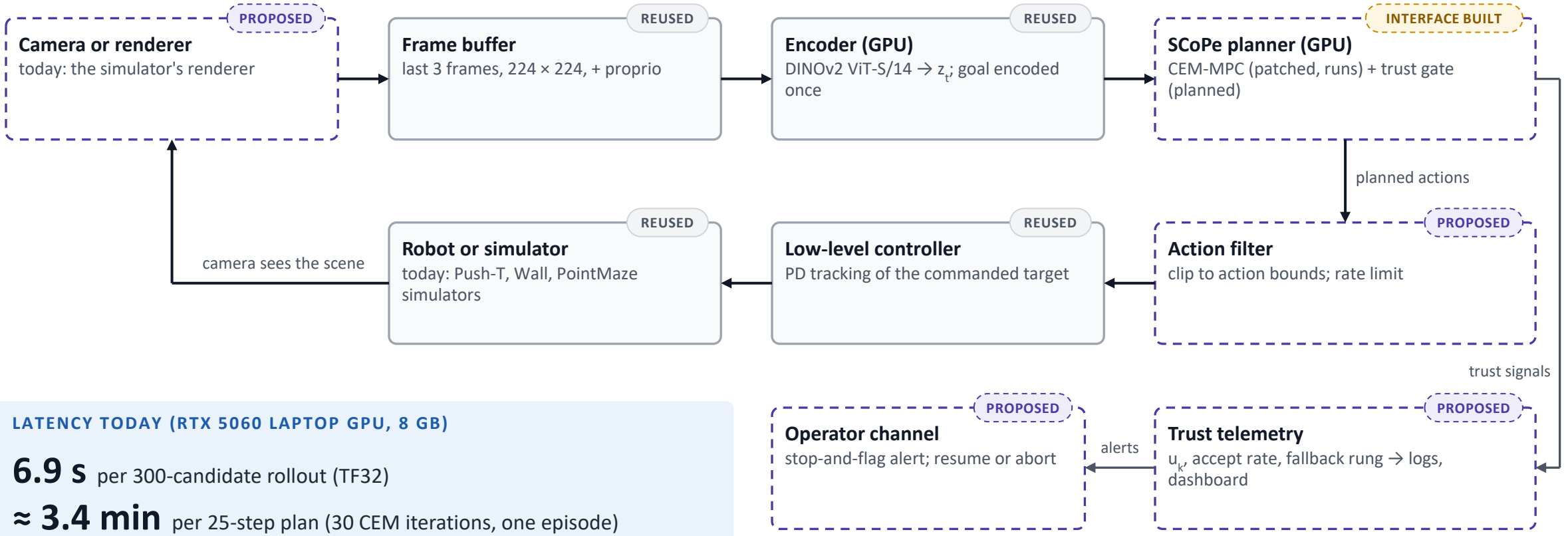
Data flow: from archives to reported numbers, with storage boundaries



Storage: E: holds code, active data, logs and summaries (about 18 GB free); F: holds archives, caches and latent shards. Upstream data is size-checked before unpacking (Wall unpacks to 55 GB, so only its 192 validation episodes are kept).

PROPOSED MEASURED

Deployment view (proposed): what inference on a robot would need



LATENCY TODAY (RTX 5060 LAPTOP GPU, 8 GB)

6.9 s per 300-candidate rollout (TF32)

≈ 3.4 min per 25-step plan (30 CEM iterations, one episode)

Too slow for real-time control. Levers: fewer iterations (Push-T success plateaus by about iteration 22), fewer samples, a faster GPU. V-JEPA 2-AC reports about 16 s per action on its hardware [2].

Runs today, in simulation: renderer → frame buffer → encoder → CEM-MPC → the simulator's PD controller. Everything tagged PROPOSED is design only.

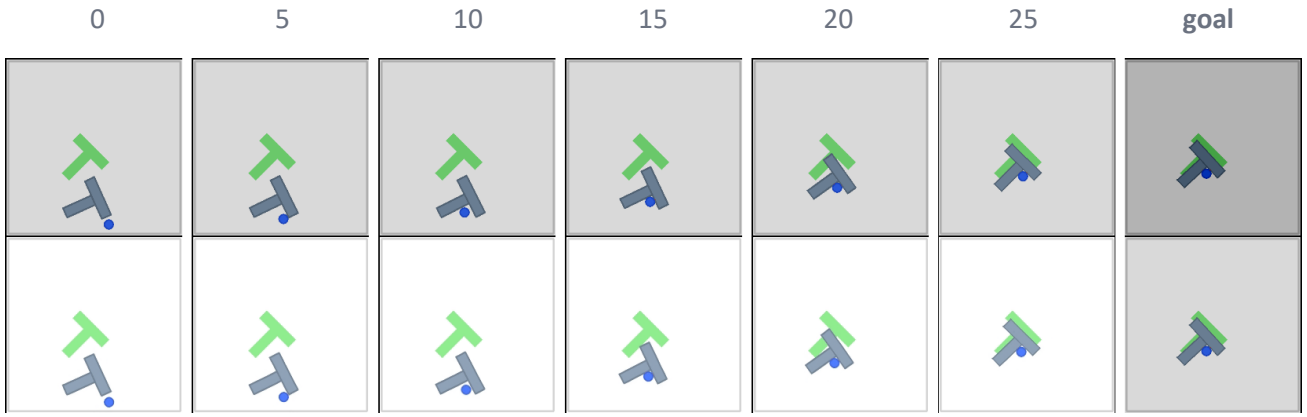
Implementation status: what exists today and what is planned

Layer	Component (actual tools and modules)	Status	Evidence in the repository
Compute	RTX 5060 Laptop GPU (8 GB) · WSL2 Ubuntu · Python 3.10 · PyTorch 2.7.1 + CUDA 12.8	BUILT	setup_wsl.sh, requirements.txt
World model	DINO-WM @ 0a9492f (git submodule) + released checkpoints, SHA-256-checked	REUSED	external/dino_wm
Simulators	pymunk (Push-T) · custom 2-D (Wall) · MuJoCo 2.1 + mujoco-py (PointMaze)	REUSED	Phase 0 run folders
Runtime patches	fused attention · encoder dedup · chunked rollouts · TF32 · no-grad encode · serial simulators	BUILT	dino_wm_patches.py
Interfaces	LatentWorldModel · LatentRollout · TrustScorer · TrustGate · GateDecision	INTERFACES	*/interface.py
Experiment ops	per-env run scripts · detached launcher · log summariser · metrics extractor	BUILT	scripts/
DINO-WM adapter	implements LatentWorldModel; encodes each observation once for all 300 candidates	PLANNED	Phase 1
Error logger	imagined vs real latents per candidate → .npz shards + JSON index	PLANNED	design doc §2, Phase 1
Trust scorers	ensemble · MC-dropout · kNN latent density · self-consistency · cost margin	PLANNED	design doc §3, Phase 2
Gated control loop	step-wise MPC with gate and fallback (upstream MPC runs all 5 actions)	PLANNED	design doc §5, Phase 3
Second backbone	V-JEPA 2-AC adapter behind the same interface	STRETCH	—

Built = our code, working and used in the Phase 0 runs. Interfaces = typed contracts only, no implementation yet. Unit tests are not written yet (tests/ is empty).

MEASURED

Demo: the baseline planner runs end to end on our hardware



Push-T · seed 100 · episode 3, simulator steps 0–25 and goal. Top: simulator; bottom: decoded imagination. They agree, and the episode is solved in round 1.

WHAT WORKS TODAY

- ✓ Released checkpoints load and plan on an 8 GB laptop GPU (WSL2, PyTorch 2.7.1)
- ✓ Push-T: 3 seeds × 50 episodes × 3 MPC rounds completed (about 3.5–4.5 h per seed)
- ✓ Wall: 3 seeds × 50 episodes × 3 rounds. PointMaze: 50 episodes, all solved after round 2
- ✓ Per-iteration success logging, rollout plots, and metrics regenerated by script
- ✓ Live demo: a 5-episode smoke test runs in about 4 minutes (command in the notes)

run log, Push-T seed 99 (verbatim excerpt; … marks skipped lines)

```
[scope] DINO-WM patches: fused attention, encoder dedup,
        chunk 50, tf32, skip_solved=True
[scope] simulators: serial, in-process
[scope] CEM: planning 50 of 50 episodes (0 already
        solved, skipped); GPU holds 298 MiB before
        the first rollout
Success rate:  0.08
[scope] CEM step 1/30: 343 s, GPU peak 2048 MiB
Success rate:  0.18
[scope] CEM step 2/30: 333 s, GPU peak 2048 MiB
…
Success rate:  0.9
[scope] CEM step 30/30: 333 s, GPU peak 2048 MiB
MPC iter 0 Eval -------
Success rate:  0.9
[scope] CEM: planning 5 of 50 episodes (45 already
        solved, skipped); GPU holds 298 MiB before
        the first rollout
```

outputs/runs/phase0/pusht/full_20261010_005356.log

MEASURED TARGET

Benchmark design: same pairs, same budget, four controls

Metric	Definition	Better	Tests
Success per round	share of the 50 start/goal pairs solved after MPC round r (simulator success test)	higher	H4
Calibration	AUROC / AUPRC of u_k for "real latent error in the top 10%"; Spearman ρ; expected calibration error of binned u_k	higher ρ, AUROC; lower ECE	H1
Gate errors	FRR = P(reject | real rollout succeeds); FAR = P(accept | real rollout fails)	lower	H2, H3
Compute	CEM iterations, wall-clock and fallback activations per episode	lower	cost
Latent error (label)	e_k = latent distance between imagined $\hat{z}_{t+k}$ and the encoding of the real frame	—	ground truth

BASELINES AND CONTROLS

B0 Plain CEM-MPC — MEASURED
DINO-WM, upstream settings

B1 Soft penalty — PLANNED
cost + λ · Σ u_k (MOPO-style [9])

B2 Random gate — PLANNED
same rejection rate as SCoPe

B3 Oracle gate — PLANNED
rejects on the true latent error: the ceiling

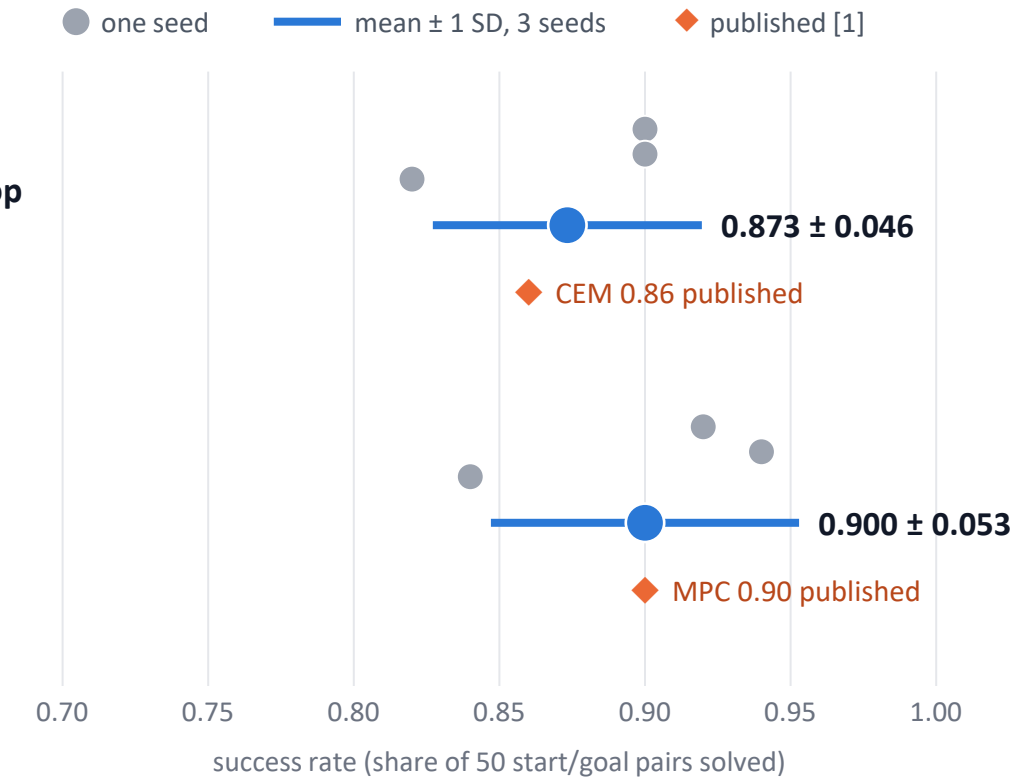
B4 SCoPe gate — PLANNED
$u_k \le \tau$ with the fallback ladder

PROTOCOL

- Push-T first (3 seeds × 50 start/goal pairs), then Wall and PointMaze. Identical seeds, pairs and CEM budget (300 samples, top 30, 30 iterations, horizon 5) for every method.
- Paired comparisons on the same pairs; mean and spread over seeds; bootstrap 95% CIs over episodes. At n = 50 and p ≈ 0.9 the binomial SE is about 0.04, so unpaired differences under ~0.1 are within noise.

MEASURED PUBLISHED

Our reproduction matches DINO-WM on Push-T within seed-to-seed noise



One CEM plan, executed open-loop
MPC round 1

seeds 99 / 100 / 101: 0.90 / 0.90 / 0.82

With replanning
after MPC round 3

seeds 99 / 100 / 101: 0.92 / 0.94 / 0.84

Round 1 executes one 5-step CEM plan before any feedback, the setting of the paper's CEM row; rounds 2–3 replan from the real state, its MPC setting. The paper does not state its replanning budget, so we report every round.

OTHER ENVIRONMENTS

Wall · mean of 3 seeds

	ours	published [1]
Round 1	**0.52**	0.74 (CEM)
Round 2	**0.91**	
Round 3	**0.95**	0.96 (MPC)

Per seed, round 1: 0.54 / 0.50 / 0.52. The round-1 gap repeats on every seed; after replanning we are close to the paper.

PointMaze · seed 99

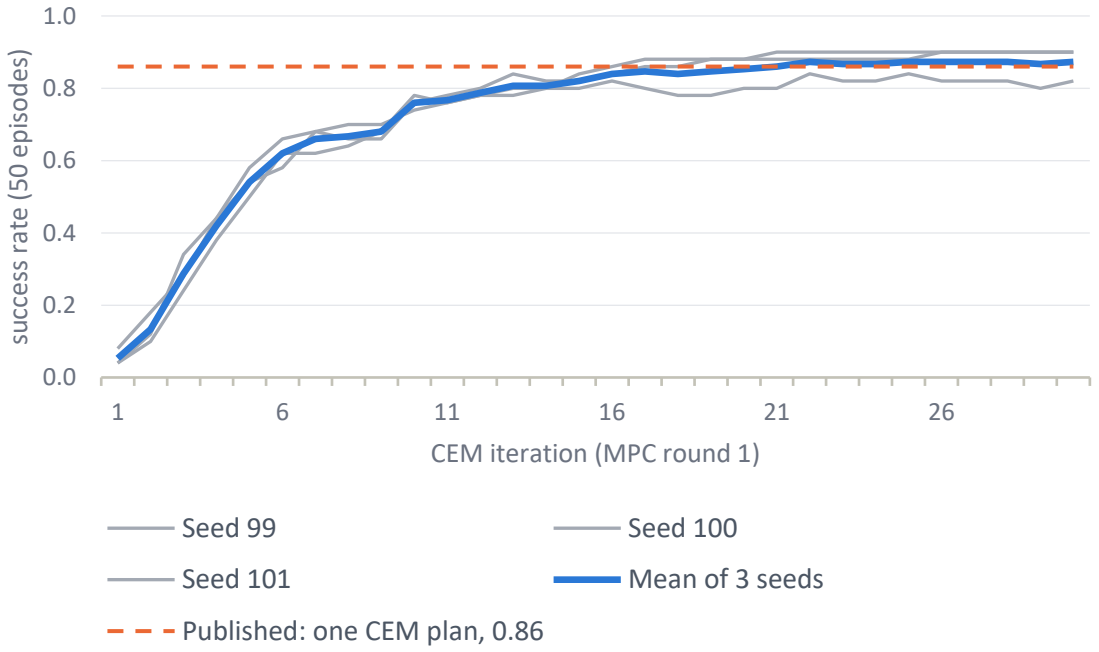
0.86 after one CEM plan and 1.00 after round 2, against the published 0.80 and 0.98.

Binomial SE at n = 50, p ≈ 0.9: about 0.04.

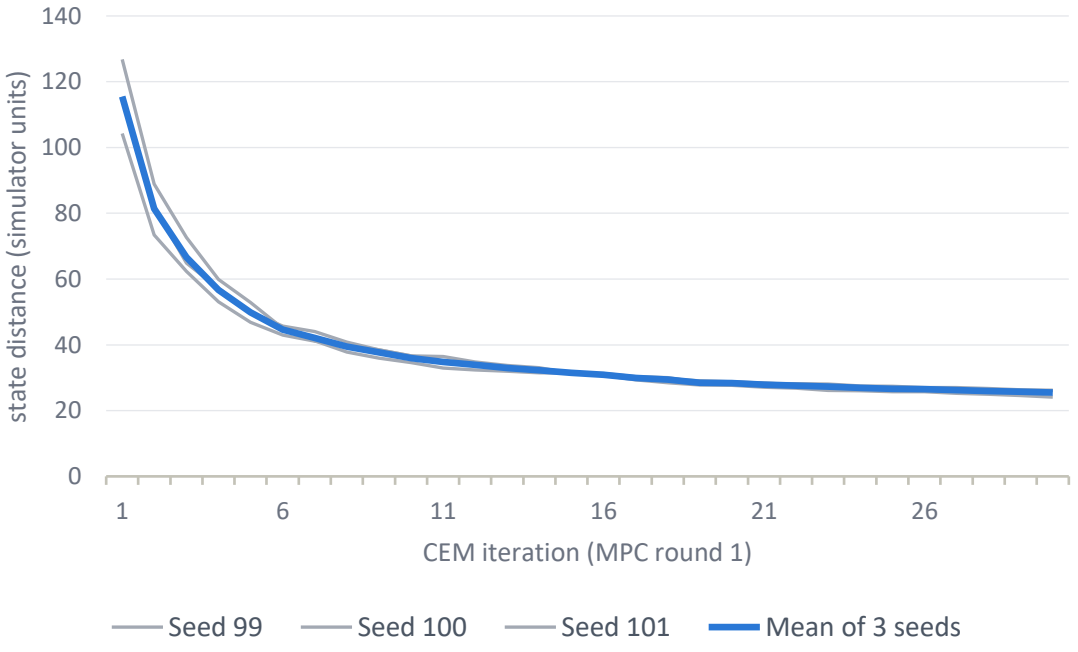
MEASURED PUBLISHED

Planning quality saturates after about 22 of 30 CEM iterations

Success rate if CEM's current plan were executed



Mean final state distance to the goal (lower is better)



0.05 → 0.76

mean success, iteration 1 → 10

0.873 = 0.873

iteration 22 vs 30: no further gain

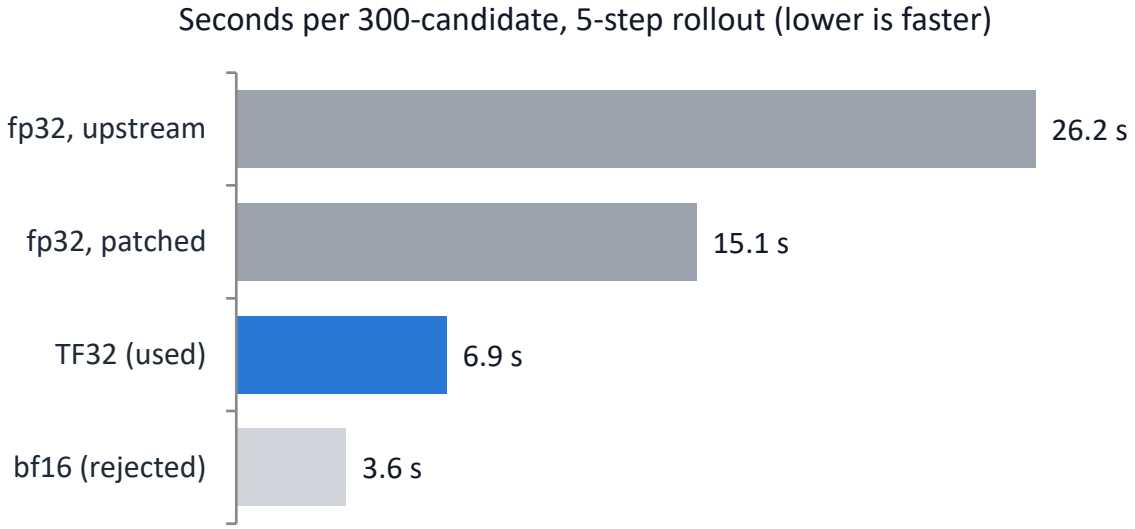
≈ 346 s

per iteration for 50 episodes (range 332–372 s)

So what: iterations 23–30 (≈ 45 min per round here) bought no success. Compute per episode is part of H4: a gate must not make CEM slower to converge.

MEASURED

Runtime patches: 3.8× faster on 8 GB, with CEM's choices unchanged



Setup	Elite overlap (σ = 1 / 0.1 / 0.02)	Latent drift	Peak VRAM
Noise floor: upstream, chunk 25	100 / 100 / 100%	0.21 / 0.11 / 0.10%	—
fp32, patched	100 / 100 / 100%	0.25 / 0.16 / 0.09%	3.0 GB
TF32, patched (used)	**100 / 100 / 100%**	0.72 / 0.38 / 0.34%	3.0 GB
bf16, patched (rejected)	**100 / 93 / 83%**	5.3 / 2.6 / 2.7%	2.3 GB

Reference: unmodified model in fp32 (26.2 s, 3.8 GB peak). Elite overlap = share of CEM's top 30 that stays the same; small σ mimics late iterations, where rankings are most fragile.

3.8×

faster per rollout than upstream fp32 (26.2 s → 6.9 s)

0%

change in CEM's update (elite-mean action) with TF32

2 h 50 min

Push-T round 1 (50 episodes × 30 iterations); ≈ 11 h estimated unpatched

298 MiB

GPU memory before the first rollout, after removing ≈ 3 GB of autograd state

MEASURED

H1 pilot: an untrained MC-dropout score flags the worst imagined steps

Score	AUROC	95% interval	Spearman ρ
MC-dropout (4 passes)	**0.863**	**0.84–0.88**	**0.65**
Action perturbation	0.657	0.64–0.67	0.27
Distance to training data	0.619	0.56–0.66	0.12
Imagined cost	0.511	0.47–0.55	0.04
Image perturbation	0.496	0.44–0.55	-0.17
Distance moved (control)	0.790	0.76–0.82	0.51
Random (control)	0.500	0.49–0.51	-0.00

Label: real latent error in the top 10% for its imagined step; AUROC computed per step (1–5) and averaged; bootstrap over 80 decision points (500 resamples).

SETUP

Push-T, 3 seeds, 80 start/goal pairs, released checkpoint, nothing trained. 192 candidates per pair from CEM iterations 1, 5 and 10, each executed open-loop in the simulator: 76,800 imagined steps with a real latent next to them.

OFFLINE GATE REPLAY (REJECT 30%, PICK LOWEST IMAGINED COST)

Selection	Chosen plan succeeds
Plain	71.3%
Oracle gate (true error)	72.5%
Random gate	70%
MC-dropout gate	70%
Action-perturbation gate	75%

Reading: MC-dropout is the only score whose interval clears both controls, so it passes the go / no-go bar (3 seeds). The gate replay is flat even for the oracle (one pair = 2.5 points), so error prediction alone does not yet show a gain: that is H4, in the closed loop.

ILLUSTRATIVE · MEASURED · TARGET

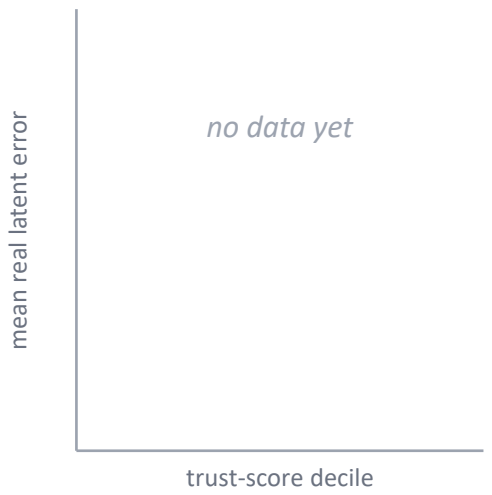
Planned figures for the final review, with pass criteria fixed now

(a) H1 · ROC of the trust score



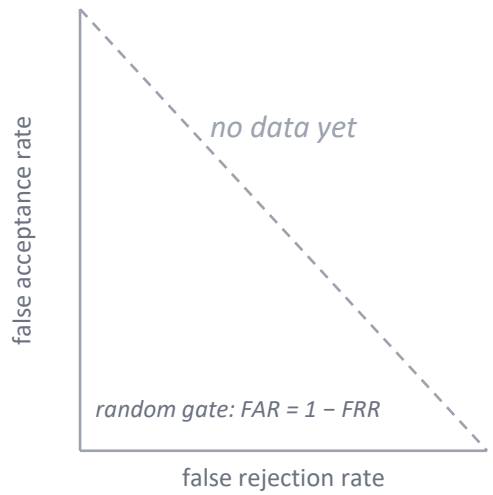
Pass: AUROC 95% CI above 0.5 (random score) and above the distance-to-start baseline.

(b) H1 · Reliability



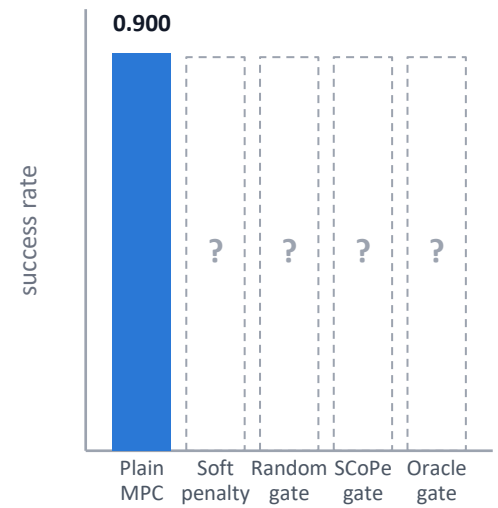
Pass: mean real error rises across score deciles (Spearman ρ > 0 with CI excluding 0).

(c) H2 / H3 · Gate error trade-off



Pass: SCoPe's curve lies below the random-gate line at matched rejection rates.

(d) H4 · Success after round 3



Pass: SCoPe above plain MPC and the random gate in a paired test (50 pairs × 3 seeds); oracle = ceiling.

Panels (a)–(c) contain no data; the dashed reference lines are what a random score or random gate would give by construction. In (d), only the blue bar is measured (plain MPC, Push-T, mean of 3 seeds); the dashed outlines are slots for planned runs.

What is new, what is reused, and what this project cannot claim

Type	Contribution	Closest existing alternative	Our difference	Status	Validation
EXPERIMENTAL	**Error-labelled imagined-vs-real latent dataset**	Biased Dreams [17] (RSSM ensembles); ACPC [19] (perturbation consistency)	frozen-encoder JEPA patch latents; real rollouts per candidate; latent error and task consequence	PLANNED · P1	coverage per context; label consistency across seeds
EXPERIMENTAL	**Calibration test of 5 candidate trust scores (H1–H3)**	ensembles [22], MC-dropout [23] in state-space MBRL [6, 9, 10]	scored per imagined step against real multi-step error, with random and trivial controls	PLANNED · P2	AUROC/AUPRC, ρ, ECE, FRR/FAR with bootstrap CIs
NEW COMBINATION	**Decision-time trust gate with a fallback ladder in CEM-MPC**	MOReL HALT [10], MACURA [11] (training time); latent safety filter [16] (safety sets)	vetoes goal-reaching candidates at planning time; no world-model retraining; explicit stop-and-flag	PLANNED · P3; interfaces built	H4 vs plain MPC, soft penalty, random and oracle gates
ADAPTATION	**8 GB reproduction of DINO-WM planning**	upstream code on data-centre GPUs [26]	3.8× faster runtime patches, verified to keep CEM's decisions	BUILT, measured	Push-T 3 seeds vs paper; fidelity benchmark
ENGINEERING	**Backbone-agnostic interfaces**	per-model planner code (DINO-WM, V-JEPA 2-AC)	one contract; imagined steps typed apart from observed context	BUILT (contracts)	second backbone adapter (stretch)

REUSED AS-IS: DINO-WM world model and checkpoints, CEM-MPC, simulators, datasets [1, 26]. FUTURE: V-JEPA 2-AC backbone; outcome-grounded scores [18, 19].

LIMITATIONS AND HONEST ASSESSMENT

- Evidence so far is a reproduction; it does not yet support any SCoPe hypothesis.
- Simulated 2-D tasks only; no real robot, so no real-world claims.
- One backbone so far; V-JEPA 2-AC is a stretch goal on 8 GB.
- 3 seeds × 50 episodes: about ±0.04 binomial SE per rate.
- Trust scores may fail to track error [17, 18]; H1 can fail, and we will report it.
- Upstream MPC uses the simulator's success flag to stop; we keep it for all methods.

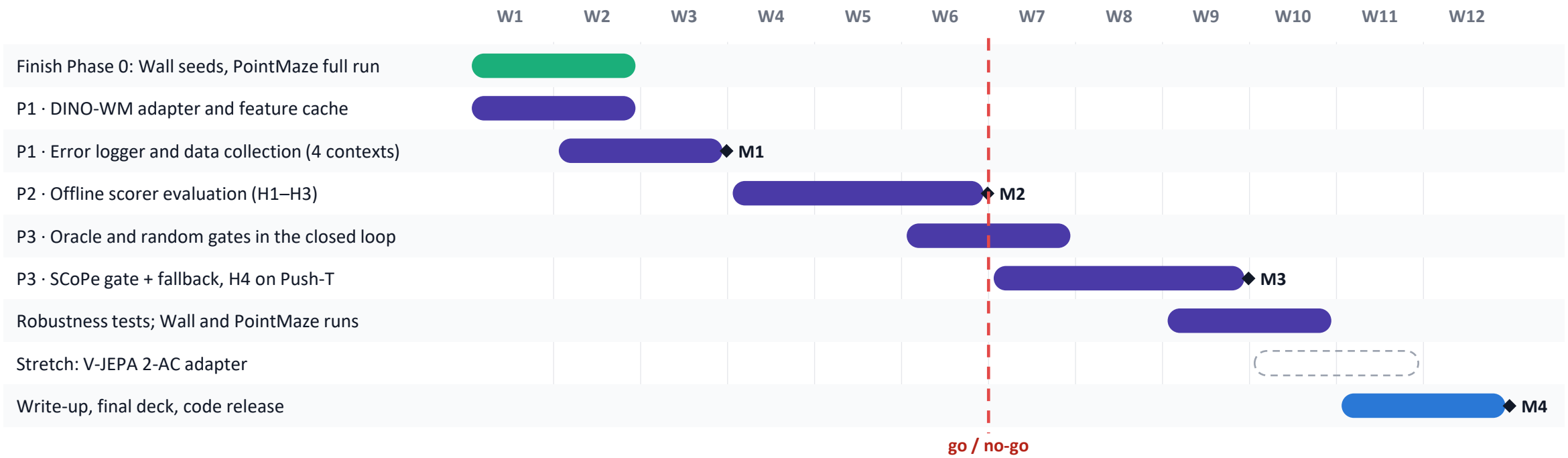
Progress: baseline and design complete; the SCoPe experiments are next

Work package	Status	Evidence	Remaining	Depends on / blocker
Requirements and problem definition	Done	design doc: H1–H4, labels, controls	fix the real-rollout budget	—
Literature review	Done	26 verified references; 13 studies compared	read ACPC and Beyond Task Success in full	—
Environment and data pipeline	Done	checksummed downloads; 3 simulators run	encoder-feature cache for ensembles and kNN	disk space: F: drive
MDP formulation	Done	slide 14, matched to code and configs	formal spec of the constrained variant	—
Algorithm implementation	Partial	baseline CEM-MPC runs with patches; interfaces	adapter, scorers, gate, fallback loop	—
Training pipeline	Not started	world model reused, nothing to train yet	ensemble heads on cached features	feature cache
Evaluation	Partial	Push-T 3 seeds; PointMaze 1 seed; Wall 3 seeds; H1 pilot	H1 at scale; H2–H4	GPU time for more seeds
Interface and visualisation	Partial	rollout plots; log parsers; this deck's charts	trust telemetry and reliability plots	P2 outputs
Documentation	Partial	setup, Phase 0 report, Phase 1–2 design	API docs, final report	—
Final validation	Not started	—	H4 closed-loop runs, robustness tests	P1–P3

4 done · 4 partial · 2 not started (unweighted count of the 10 review work packages). We do not turn this into a percentage: the remaining packages hold most of the research effort.

TARGET

Roadmap (proposed, relative weeks): three gated phases



M1 error-labelled dataset · M2 H1–H3 report + go/no-go: continue to the closed loop only if a score beats the random and distance-to-start baselines on H1 (non-overlapping 95% CIs); otherwise report the negative result with the oracle-gate ceiling · M3 H4 results · M4 final report

TOP RISKS → MITIGATION

No score tracks error (H1 fails) → oracle gate shows the ceiling; report it; try outcome-grounded scores.
Compute: 2 h 50 min per Push-T round → cached features; fewer CEM iterations for development runs.
Gate over-rejects → calibrate τ on held-out points; sweep τ; report fallback rates.

DONE WHEN

- H1–H3 reported for ≥ 3 scores with bootstrap CIs
- H4: SCoPe vs 4 controls on the same 50 pairs × 3 seeds
- every number regenerated from logs by script

Where we stand: baseline reproduced, trust test designed and ready

THE PROBLEM

Latent planners act on imagined futures with no reliability check. We logged a case where imagination and reality diverge (Push-T, seed 101, episode 0).

EVIDENCE SO FAR

DINO-WM planning reproduced on an 8 GB laptop GPU: Push-T 0.873 ± 0.046 (one plan) and 0.900 ± 0.053 (with replanning) over 3 seeds, vs 0.86 / 0.90 published; 3.8× faster with identical CEM decisions.

MEASURED

NEXT MILESTONE

M1: an error-labelled dataset of imagined vs real latents, then the offline H1–H3 test with a go/no-go gate before the closed-loop H4 experiment.

TARGET

Contribution, stated narrowly: a tested answer to whether decision-time trust scores on JEPA latents make latent planning more reliable, including if the answer is no.

Questions

[TEAM MEMBERS] · [GUIDE NAME] · [INSTITUTION]

References [1]–[13]

[1] G. Zhou, H. Pan, Y. LeCun, L. Pinto. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. ICML 2025, PMLR 267:79115–79135. arxiv.org/abs/2411.04983

[2] M. Assran et al. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. arXiv:2506.09985, 2025. arxiv.org/abs/2506.09985

[3] M. Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. TMLR, 2024. arxiv.org/abs/2304.07193

[4] M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA). ICCV 2023. arxiv.org/abs/2301.08243

[5] Y. LeCun. A Path Towards Autonomous Machine Intelligence. Position paper, v0.9.2, 2022 (not peer-reviewed). openreview.net/forum?id=BZ5a1r-kVsf

[6] K. Chua, R. Calandra, R. McAllister, S. Levine. Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models (PETS). NeurIPS 2018. arxiv.org/abs/1805.12114

[7] D. Hafner et al. Learning Latent Dynamics for Planning from Pixels (PlaNet). ICML 2019, PMLR 97:2555–2565. arxiv.org/abs/1811.04551

[8] M. Janner, J. Fu, M. Zhang, S. Levine. When to Trust Your Model: Model-Based Policy Optimization (MBPO). NeurIPS 2019. arxiv.org/abs/1906.08253

[9] T. Yu et al. MOPO: Model-based Offline Policy Optimization. NeurIPS 2020. arxiv.org/abs/2005.13239

[10] R. Kidambi, A. Rajeswaran, P. Netrapalli, T. Joachims. MOReL: Model-Based Offline Reinforcement Learning. NeurIPS 2020. arxiv.org/abs/2005.05951

[11] B. Frauenknecht, A. Eisele, D. Subhasish, F. Solowjow, S. Trimpe. Trust the Model Where It Trusts Itself: Model-Based Actor-Critic with Uncertainty-Aware Rollout Adaption (MACURA). arXiv:2405.19014, 2024. arxiv.org/abs/2405.19014

[12] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. Mastering Diverse Control Tasks through World Models (DreamerV3). Nature 640:647–653, 2025. doi.org/10.1038/s41586-025-08744-2

[13] N. Hansen, H. Su, X. Wang. TD-MPC2: Scalable, Robust World Models for Continuous Control. ICLR 2024. arxiv.org/abs/2310.16828

References [14]–[26]

[14] V. Sobal et al. Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models (PLDM). NeurIPS 2025. arxiv.org/abs/2502.14819

[15] W. Zhang et al. Hierarchical Planning with Latent World Models (HWM). arXiv:2604.03208, 2026. arxiv.org/abs/2604.03208

[16] J. Seo, K. Nakamura, A. Bajcsy. Uncertainty-aware Latent Safety Filters for Avoiding Out-of-Distribution Failures. CoRL 2025. arxiv.org/abs/2505.00779

[17] J. Berger, B. Frauenknecht, S. Trimpe, B. Leibe. Biased Dreams: Limitations to Epistemic Uncertainty Quantification in Latent Dynamics Models. RLC 2026. arxiv.org/abs/2604.25416

[18] Z. Wang. Learning from World Feedback: Why Model Uncertainty Fails as a Risk Signal in Model-Based RL. ICML 2026 Workshop on RL from X Feedback. arxiv.org/abs/2607.16591

[19] G. An et al. Diagnosing JEPA World Models with Action-Conditioned Predictive Consistency (ACPC). arXiv:2608.12939, 2026. arxiv.org/abs/2608.12939

[20] J. Hu et al. Representation Geometry Matters for Planning with JEPA World Models. arXiv:2608.16287, 2026 (arXiv title spells "Reperesentation"). arxiv.org/abs/2608.16287

[21] G. Lee, B.-T. Zhang. Beyond Task Success: Stage-Wise Reliability of World Model Planning under Sensing Degradation. arXiv:2609.07126, 2026. arxiv.org/abs/2609.07126

[22] B. Lakshminarayanan, A. Pritzel, C. Blundell. Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. NIPS 2017. arxiv.org/abs/1612.01474

[23] Y. Gal, Z. Ghahramani. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. ICML 2016. arxiv.org/abs/1506.02142

[24] C. Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. RSS 2023 (origin of the Push-T task used by DINO-WM). arxiv.org/abs/2303.04137

[25] J. Fu, A. Kumar, O. Nachum, G. Tucker, S. Levine. D4RL: Datasets for Deep Data-Driven Reinforcement Learning. arXiv:2004.07219, 2020 (PointMaze). arxiv.org/abs/2004.07219

[26] DINO-WM code (MIT licence), pinned at commit 0a9492f; datasets and checkpoints on OSF (project bmw48). github.com/gaoyuezhou/dino_wm

APPENDIX A1

Deviations from the upstream DINO-WM setup, and their risk to fidelity

Change	Reason	Risk to fidelity
PyTorch 2.3.0 → 2.7.1 (CUDA 12.8)	RTX 5060 (sm_120) has no kernels in 2.3	numerical drift; low
Python 3.9 → 3.10	shared environment with the scope package	low
mujoco-py built with system gcc, -std=gnu17, pointer-type errors as warnings	2021-era generated C fails on gcc 14+	none (compile flags only)
Lean dependency set (no TF, JAX, robosuite)	unused on the planning path	none if imports resolve
wandb disabled, git lookup skipped	no W&B project; git probe fails on /mnt/e	none
Runtime patches: fused attention, encoder dedup, chunked rollouts, TF32	upstream needs > 8 GB and 26 s per 300-candidate rollout here	latent drift ≤ 0.7%; identical CEM elites and updates in every benchmark case
PyTorch capped at 75% of VRAM	Windows silently spills GPU overflow into system RAM	none; overflow raises an error instead
TORCH_FORCE_NO_WEIGHTS_ONLY_LOAD=1	torch ≥ 2.6 refuses checkpoints that pickle whole modules	none; restores torch 2.3 loading of the checksummed official files
MPC capped at 3 rounds; serial in-process simulators	upstream max_iter is unbounded; 50 forked simulators exhaust WSL memory	paper's budget unstated, so every round is reported

APPENDIX A2

Glossary of terms and abbreviations used in this deck

JEPA joint-embedding predictive architecture: predicts representations, not pixels [4, 5]

DINOv2 self-supervised vision transformer; its patch features are DINO-WM's frozen encoder [3]

Latent z the encoder's representation of an image: 196 patch tokens × 384 numbers

World model learned model predicting the next latent from the current one and an action

MDP / POMDP (partially observable) Markov decision process

MPC model predictive control: plan a short horizon, execute, observe, replan

CEM cross-entropy method: sample, keep the best (elites), refit, repeat

Elites the top-k candidates by cost in one CEM iteration (k = 30 here)

MPPI model predictive path integral: averages all samples weighted by cost

RSSM recurrent state-space model (PlaNet, Dreamer)

Trust score u_k per-imagined-step unreliability estimate; higher = less trusted

τ, λ rejection threshold of the gate; weight of the soft-penalty control

AUROC / AUPRC area under the ROC / precision-recall curve

ECE expected calibration error of binned scores

FRR / FAR false rejection / false acceptance rate of the gate

Spearman ρ rank correlation between score and real error

OOD out of distribution: unlike the training data

Proprio proprioceptive input: the agent's own position

Frameskip simulator steps per planned action (5 here)

TF32 / bf16 reduced-precision GPU number formats

SDPA scaled dot-product attention (fused GPU kernel)

OSF / WSL2 Open Science Framework (data host) / Windows Subsystem for Linux